COIMBRA INSTITUTE OF ENGINEERING

and

Altice Labs

Departamento de Engenharia Informática e de Sistemas
Licenciatura em Engenharia Informática

Exploração do OpenTelemetry e Implementação de uma Prova de Conceito

Relatório de Progresso Quinzenal

Bernardo Arede 2022138769

Orientadores:
Ricardo Filipe, Altice Labs
Ana Alves, ISEC

10 de março de 2026

1 Estado da Arte

1.1 Desafio em arquiteturas distribuídas

Nas últimas décadas, o paradigma de desenvolvimento de software passou de arquiteturas monolíticas para ecossistemas baseados em microserviços e aplicações cloud-native. Com o crescimento destas arquiteturas distribuídas, a observabilidade tornou-se um fator crítico para garantir não só o desempenho e a fiabilidade, mas também a rápida deteção e correção de falhas.

Num sistema monolítico, o fluxo de execução é linear e ocorre num único domínio, o que facilita o isolamento de erros. Contudo, em ambientes modernos, um único pedido de um utilizador pode atravessar dezenas de microserviços diferentes, desenvolvidos em linguagens diferentes e alojados em múltiplas arquiteturas. Neste contexto, as ferramentas tradicionais de monitorização revelam imensas limitações quando aplicadas a sistemas complexos e altamente distribuídos. A monitorização clássica, focada apenas na verificação de estados de alerta, tornou-se insuficiente para responder à verdadeira questão no contexto de sistemas distribuídos: "Porquê e onde é que o sistema está a falhar?".

1.2 Paradigma da Observabilidade

Para dar resposta a esta complexidade, a indústria adotou o conceito de Observabilidade. Ao contrário da monitorização tradicional, que é reativa, a observabilidade é uma propriedade do sistema que permite inferir o seu estado interno a partir das suas saídas externas (telemetria). Este conceito apoia-se convencionalmente em três pilares fundamentais de dados:

- **Métricas (Metrics):** Representações numéricas de dados agregados ao longo do tempo (ex.: taxa de uso de CPU, número de pedidos por segundo). São cruciais para a deteção rápida de anomalias do sistema.
- **Traces:** Representam o ciclo de vida completo de um pedido à medida que este viaja pelos vários componentes de um sistema distribuído. Permitem identificar problemas de performance e mapear dependências entre serviços [1].
- **Logs:** Registos imutáveis e discretos de eventos que ocorreram no sistema. Oferecem o contexto de mais alto nível de detalhe (*debugging*) quando um erro específico é detetado [5].

1.3 OpenTelemetry

Historicamente, a recolha de sinais de telemetria sofria de uma forte dependência de fornecedores. Cada ferramenta de monitorização exigia a instalação de agentes proprietários ou a integração de bibliotecas específicas no código-fonte, o que elevava o custo técnico e limitava a flexibilidade.

Para combater este problema, o OpenTelemetry surge como uma solução unificada e independente de fornecedor, sendo hoje amplamente adotada pela indústria. Trata-se de um projeto da Cloud Native Computing Foundation (CNCF) que fornece um conjunto de APIs, SDKs e ferramentas padrão para a instrumentação, recolha e exportação de dados de telemetria [2]. Ao padronizar a

instrumentação e permitir a integração com diversas plataformas de monitorização, o OpenTelemetry separa a recolha dos dados da sua camada de armazenamento e visualização. Isto contribui diretamente para a redução da complexidade técnica e para a melhoria da eficiência operacional das equipas.

1.4 O Ecossistema de Análise e Visualização

A arquitetura proposta neste estágio integra o OpenTelemetry com algumas das ferramentas open-source mais consolidadas no mercado, garantindo um pipeline de observabilidade robusto:

- **Prometheus:** O padrão de facto para o armazenamento em séries temporais e alerta de métricas em ambientes cloud-native.
- **Jaeger:** Plataforma especializada na receção, armazenamento e visualização avançada de rastreios distribuídos [1].
- **Grafana e Loki:** O Grafana atua como a interface unificada para a correlação visual de todos os sinais, enquanto o Loki é otimizado para a agregação de logs com alta eficiência, partilhando sinergias com o modelo de labels do Prometheus.

A interligação destas tecnologias com serviços desenvolvidos em linguagens como Golang, orquestrada pelo OpenTelemetry Collector, forma a base tecnológica que sustenta os objetivos práticos deste projeto.

2 Zero-code Implementation e Auto-instrumentação

2.1 O Paradigma Low-code na Observabilidade

Em alinhamento com os objetivos propostos para este estágio, explorou-se a adoção de abordagens *low-code* e *zero-code* na implementação da observabilidade. Tradicionalmente, a instrumentação de aplicações exigia alterações significativas no código-fonte para injetar chamadas a bibliotecas de monitorização, um processo moroso e com elevado risco de introdução de erros em ambientes de produção.

A abordagem *zero-code* elimina esta barreira. Utilizando agentes dinâmicos ou tecnologias ao nível do kernel, torna-se possível extrair telemetria rica (métricas e traces) de uma aplicação sem alterar uma única linha do seu código original.

2.2 Mecanismos de Zero-Code no OpenTelemetry

O ecossistema OpenTelemetry suporta nativamente a auto-instrumentação para diversas linguagens de programação, incluindo Golang (uma das tecnologias a utilizar neste projeto) [4]. Este mecanismo funciona através das bibliotecas padrão e *frameworks* mais populares (como HTTP clients, drivers de base de dados e servidores web) [3].

Ao iniciar a aplicação com o agente do OpenTelemetry acoplado, este interceta automaticamente as requisições, gerando os *spans* correspondentes, propagando o contexto distribuído e exportando os dados no formato OTLP para o Collector.

2.3 Vantagens para o Ecossistema Altice Labs

A validação desta prova de conceito tem um impacto direto no contexto empresarial da Altice Labs. A capacidade de implementar observabilidade *zero-code* oferece vantagens estratégicas e operacionais consideráveis:

- **Adoção sem atrito:** Equipas com serviços atribuídos (*legacy*) podem obter visibilidade imediata sem necessitarem de alocar tempo de desenvolvimento para refatorização de código.
- **Padronização imediata:** Garante que todos os serviços exportam telemetria com as mesmas *Semantic Conventions* do OpenTelemetry [2], facilitando a correlação no Grafana.
- **Redução do débito técnico:** Separa a lógica de negócio da lógica de monitorização, mantendo os repositórios de código mais limpos e fáceis de manter.

3 Arquitetura da Prova de Conceito

3.1 Visão Geral do Sistema

A definição da arquitetura do protótipo visa estabelecer um fluxo de dados de telemetria fiável, escalável e agnóstico em relação à infraestrutura subjacente. O desenho da solução assenta num modelo produtor-consumidor, onde os serviços geram sinais de observabilidade que são centralizados, processados e posteriormente encaminhados para as respetivas bases de dados especializadas (*backends*).

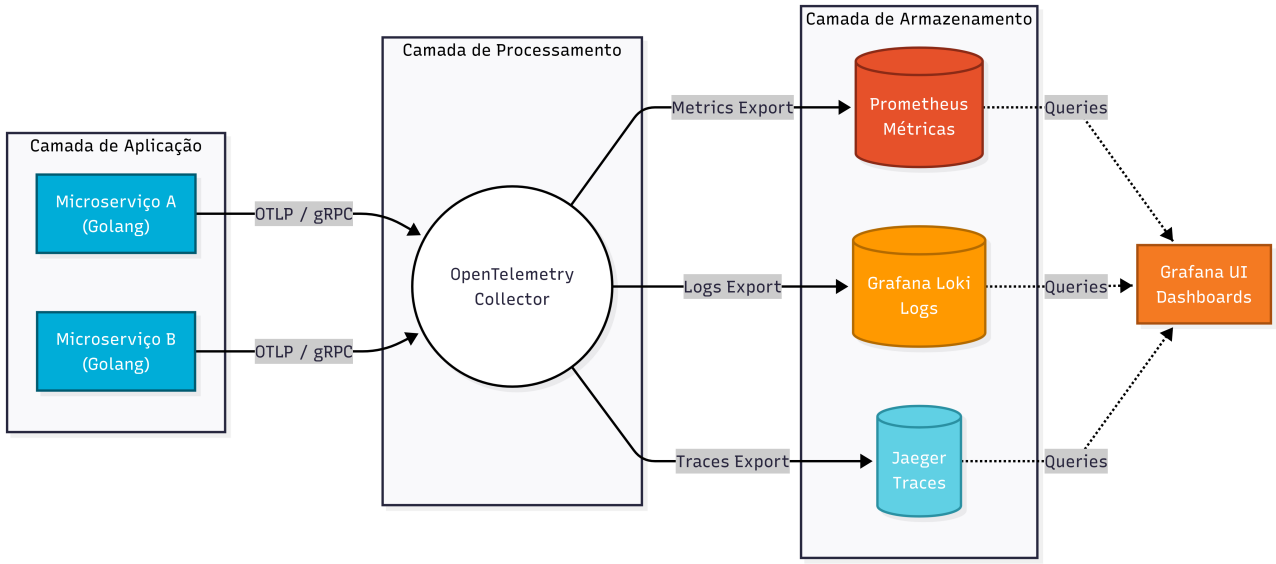

Figura 1: Diagrama da Arquitetura de Observabilidade com OpenTelemetry.

3.2 Componentes e Tecnologias

Para cumprir os requisitos de integração e padronização, a infraestrutura foi dividida em três camadas lógicas principais, suportadas por um conjunto de tecnologias *open-source*:

- **Camada de Aplicação (Geração):** Dois microserviços desenvolvidos em **Golang** que simulam transações de negócio. Estes serviços utilizam os SDKs do OpenTelemetry para instrumentar o código e exportar métricas, logs estruturados e *traces*.
- **Camada de Processamento (Pipeline):** Materializada pelo **OpenTelemetry Collector**, que atua como o *gateway* único de telemetria da rede.
- **Camada de Armazenamento e Visualização (*Backends*):**
  - **Prometheus:** Receção e armazenamento em séries temporais das métricas de performance.
  - **Jaeger:** Armazenamento e indexação dos *spans* para visualização do rastreio distribuído.
  - **Grafana Loki:** Agregação de *logs* otimizada, permitindo a correlação direta através de *trace_ids*.
  - **Grafana:** Plataforma de interface de utilizador (UI) para construção de *dashboards* unificados.

3.3 O Papel Central do OpenTelemetry Collector

A decisão arquitetural de não enviar a telemetria diretamente das aplicações para os *backends* justifica-se pelos princípios de boas práticas em sistemas distribuídos. O OpenTelemetry Collector atua como um *proxy* independente, estruturado num *pipeline* de três fases:

1. **Receivers:** Configurados para aceitar dados via protocolo **OTLP** (OpenTelemetry Line Protocol) via gRPC e HTTP, garantindo uma interface de entrada padronizada.
2. **Processors:** Responsáveis pela manipulação dos dados em trânsito. Permitem o processamento em *batch* (para otimizar o uso da rede), a adição de metadados globais (ex.: nome do ambiente ou *datacenter*) e a filtragem de dados sensíveis.
3. **Exporters:** Desacoplam as aplicações do destino final dos dados. O Collector foi configurado com exportadores nativos para encaminhar as métricas para o Prometheus, os *traces* para o Jaeger e os *logs* para o Loki, traduzindo o formato OTLP para as APIs específicas de cada ferramenta, se necessário.

3.4 Justificação da Escolha do Collector

A decisão de introduzir o OpenTelemetry Collector como peça central da arquitetura baseia-se na necessidade de criar um intermediário agnóstico ao fornecedor. O Collector atua como o motor de receção (via protocolo OTLP), processamento (agregação, filtragem) e exportação dos

dados, aliviando os microserviços dessa carga de processamento e garantindo maior segurança e flexibilidade no encaminhamento da telemetria.

Referências

[1] Jaeger Authors. *Jaeger Documentation*. 2025. URL: https://www.jaegertracing.io/docs/.

[2] OpenTelemetry Authors. *OpenTelemetry Documentation*. 2025. URL: https://opentelemetry.io/docs/.

[3] OpenTelemetry Authors. *opentelemetry-go-instrumentation Repository*. 2025. URL: https://github.com/open-telemetry/opentelemetry-go-instrumentation/.

[4] OpenTelemetry Authors. *Zero-code Instrumentation for Go*. 2025. URL: https://opentelemetry.io/docs/zero-code/go/.

[5] That DevOps Guy. *How OpenTelemetry works - How to collect logs*. 2025. URL: https://www.youtube.com/watch?v=bIxt1b0GOU4.